Introduction to Computer Vision and Image Enhancement in Spatial domain

Prepared By
Dr. Khaled Mostafa ElSayed
kmelsayed@zewailcity.edu.eg

Instructor:

Dr. Khaled Mostafa ElSayed
kmelsayed@zewailcity.edu.eg

مدينة زويل
ZEWAIL CITY
ESTABLISHED 2000
INAUGURATED 2011
جامعة العلوم والتكنولوجيا
University of Science and Technology

Computer Vision

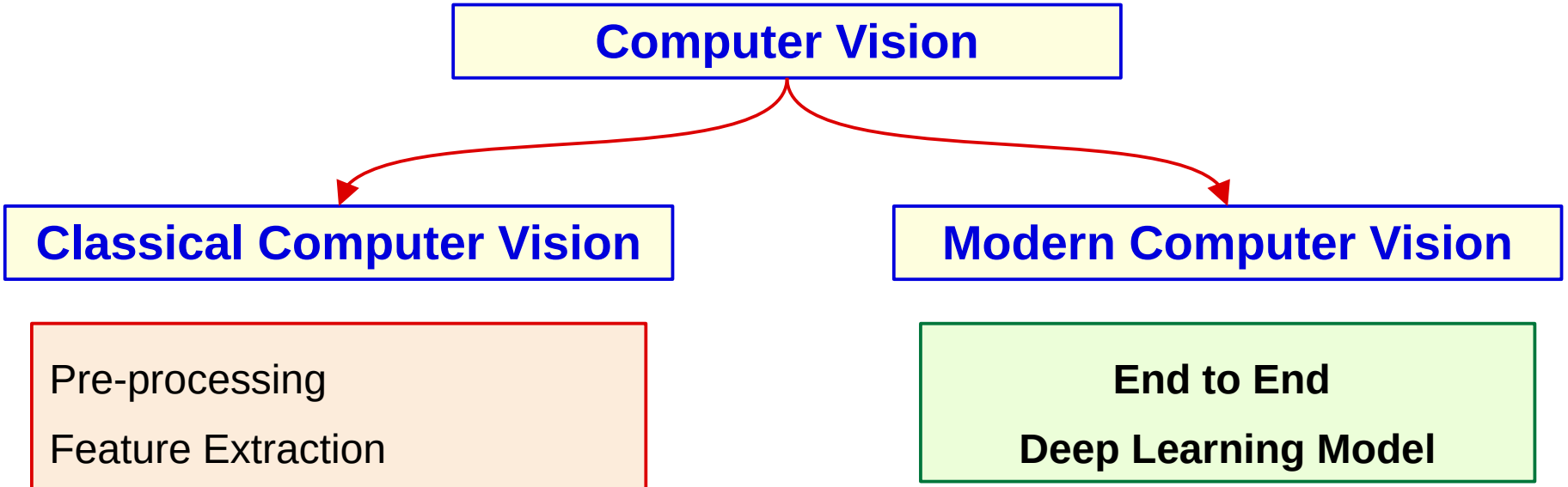
Computer vision is a field of artificial intelligence (AI) that:

- **uses** machine learning and neural networks to **teach** computers and systems to:
  - **derive** meaningful information from
  - **digital images, videos and other visual inputs**—and to
  - make **recommendations** or take **actions**

AI *enables computers to* ***think,***

Computer Vision *enables them to* ***See***, ***Observe*** *and* ***Understand***.

Computer Vision



Classical Computer Vision

Classical CV is when the features are hand crafted

Example:

Take documents photos using mobile phone then:

- Use classical CV to detect the corners and the transformations of the document
- Use classical CV to perform affine transformations and rectify the document
- Use any **traditional classifier** to perform OCR (SVM, NN, ...)

Classical Computer Vision

Pros:

- May lead to **better accuracy** for the fixed domain of data,
- **faster** inference time (on average)
- Suitable for **edge devices** (small size)
- **Explainability** is a built-in feature of many classical computer vision techniques (edge detection, corner detection, Features extraction)
- **Interpretability** is much more straightforward.
- Requires **less amount of training data**

Cons:

- Not generalizable
- Harder to maintain if data distribution changes
- Hard to carry out complex task (less accuracy)

Modern Computer Vision

End-to-End processing (e.g. using CNN)

Example:

Take documents photos using mobile phone then:

- Use **CNN (Filters)** to perform affine transformations and rectify the document
- Use **CNN (Filters)** to Extract Features
- Use **CNN (FC Layers)** to Classify Characters

Modern Computer Vision

Pros:

- **Generalization:** feature are learned by the model itself.
- **Best in outdoor** applications(uncontrolled environment) e.g. when lighting is highly variable.
- **Robust**,
- **Easier to retrain**.
- Support complex recognition task that's very hard to execute with classic methods

Cons:

- Not suitable for small datasets, (accuracy will dramatically decrease)
- Higher training time
- Higher inference time
- feature learned (automatic) may not be interpretable by humans

Some Computer Vision Tasks

- Object **Detection** and **Localization**
- Object **recognition** (object classification)
- Content-based image **retrieval** (e.g. all images similar to image given image)
- **Pose** estimation
- **OCR**
- **Facial** recognition
- **Emotion** recognition
- Human **activity recognition**
- Object **tracking**
- Image **restoration** (restore degraded images due to some external factors like lens wrong positioning, transmission interference,
- Image **inpainting** (special type of restoration) for damaged, deteriorated, or missing parts

Topic to be covered in Computer Vision Course

Classical Computer Vision

- Image Enhancement
  - **Spatial** Domain
  - **Frequency** domain
- Image **Morphology**
- Feature Extraction (Corner detection, Edge Detection, **SIFT**, …)

Modern Computer Vision

- Sliding Window
- You Only Look Once (**YOLO**)
- Regional CNN (**RCNN**- Fast RCNN – Faster RCNN)
- **Mask RCNN**
- U-Net (May Be)
- Object Tracking (May Be)

Image Enhancement

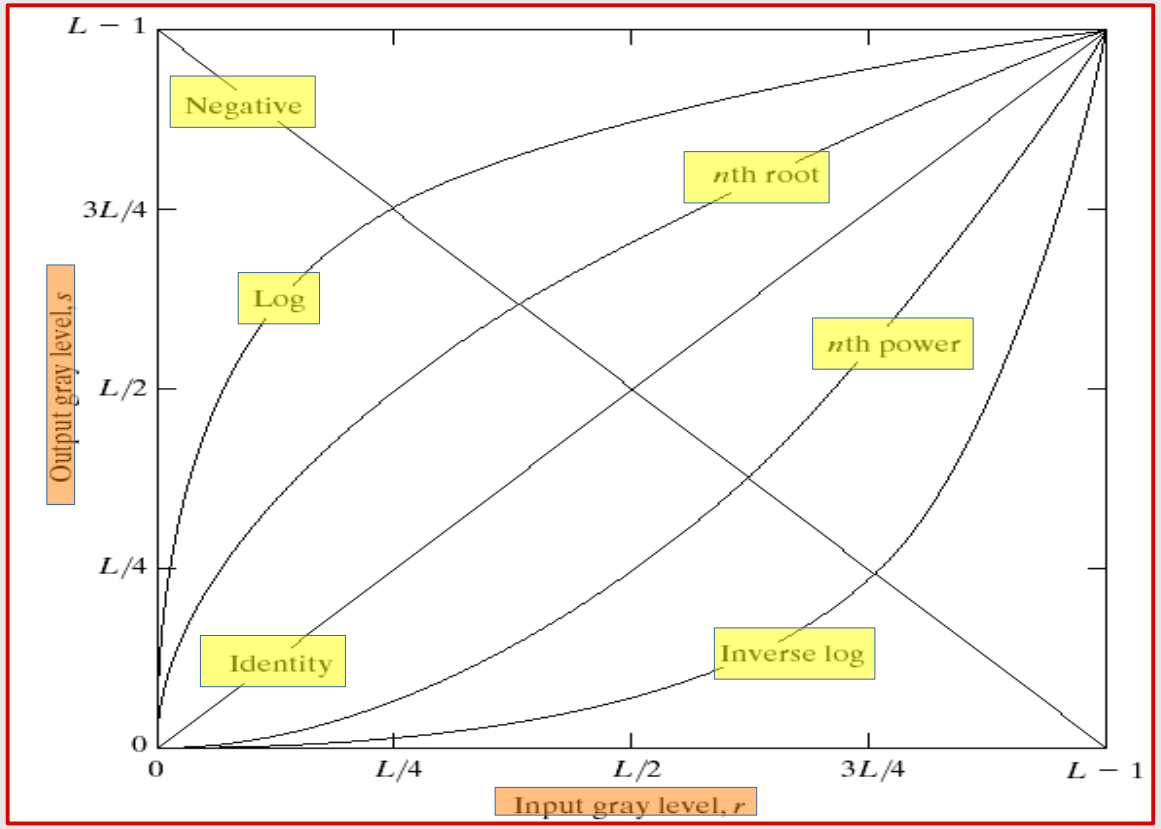
[1] Pixel Based

- Negative Transformation
- Gamma Correction
- Histogram equalization

[2] Pixels Neighbors Based (Local Enhancement)

- Spatial filtering
  - Smoothing filters
  - Ordered statistics filters
  - Sharpening filters

Gray Level Transformations of Pixel Values



For this image, what transform should we apply?

r: **Input** gray level **s**: **Output** gray level **L**: number of Gray levels =256

Image Enhancement

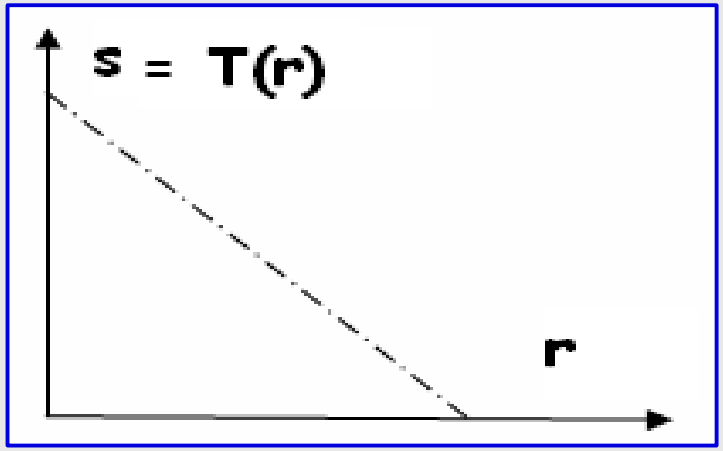
[1] Pixel Based

- **Negative Transformation**
- Gamma Correction
- Histogram equalization

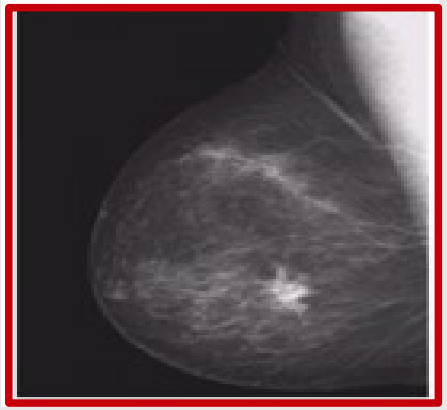
[2] Pixels Neighbors Based (Local Enhancement)

- Spatial filtering
  - Smoothing filters
  - Ordered statistics filters
  - Sharpening filters

[1] Negative Transformation



s = 255 - r



r

s

Gray Levels are from **0 (Black)** to **255 (White)**

r: **Input** gray level **s**: **Output** gray level **L**: number of Gray levels =256

Image Enhancement

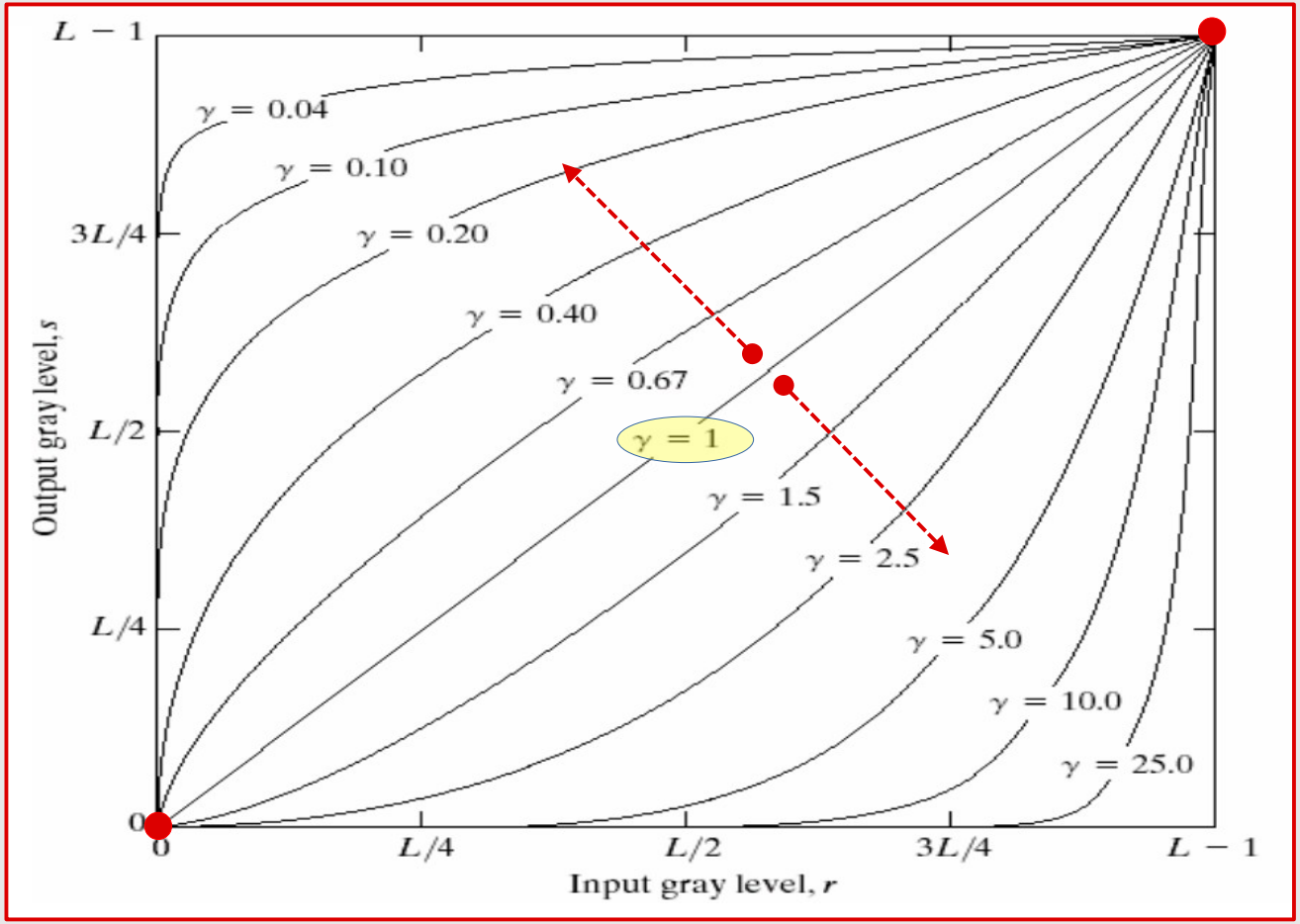
[1] Pixel Based

- Negative Transformation
- **Gamma Correction**
- Histogram equalization

[2] Pixels Neighbors Based (Local Enhancement)

- Spatial filtering
  - Smoothing filters
  - Ordered statistics filters
  - Sharpening filters

[2] Gamma Correction



$s = cr^{\gamma}$ for different γ, Let c=1

Normalize **r** before Apply Eq.
To be **from 0 to 1**

Output **S** will be normalized as well

r: **Input** gray level **s**: **Output** gray level **L**: number of Gray levels =256

[2] Gamma Correction

$s = cr^{\gamma}$ for different γ, Let c=1

	A	B	C	D	E	F	G	H	I
1		gamma	gamma	gamma	gamma	gamma	gamma	gamma	gamma
2	gray level (0-1)	0.1	0.2	0.3	0.4	2	5	10	50
3	0	0	0	0	0	0	0	0	0
4	0.05	0.741	0.549	0.407	0.302	0.003	0	0	0
5	0.1	0.794	0.631	0.501	0.398	0.01	0	0	0
6	0.15	0.827	0.684	0.566	0.468	0.023	0	0	0
7	0.2	0.851	0.725	0.617	0.525	0.04	0	0	0
8	0.25	0.871	0.758	0.66	0.574	0.063	0.001	0	0
9	0.3	0.887	0.786	0.697	0.618	0.09	0.002	0	0
10	0.35	0.9	0.811	0.73	0.657	0.123	0.005	0	0
11	0.4	0.912	0.833	0.76	0.693	0.16	0.01	0	0
12	0.45	0.923	0.852	0.787	0.727	0.203	0.018	0	0
13	0.5	0.933	0.871	0.812	0.758	0.25	0.031	0.001	0
14	0.55	0.942	0.887	0.836	0.787	0.303	0.05	0.003	0
15	0.6	0.95	0.903	0.858	0.815	0.36	0.078	0.006	0
16	0.65	0.958	0.917	0.879	0.842	0.423	0.116	0.013	0
17	0.7	0.965	0.931	0.899	0.867	0.49	0.168	0.028	0
18	0.75	0.972	0.944	0.917	0.891	0.563	0.237	0.056	0
19	0.8	0.978	0.956	0.935	0.915	0.64	0.328	0.107	0
20	0.85	0.984	0.968	0.952	0.937	0.723	0.444	0.197	0
21	0.9	0.99	0.979	0.969	0.959	0.81	0.59	0.349	0.005
22	0.95	0.995	0.99	0.985	0.98	0.903	0.774	0.599	0.077
23	1	1	1	1	1	1	1	1	1
24									

[2] Gamma Correction

Original Image

$C=1 \quad \gamma=0.6$

[2] Gamma Correction

Original Image

$C=1 \quad \gamma=0.4$

[2] Gamma Correction

Original Image

$C=1 \quad \gamma=0.3$

Image Enhancement

[1] Pixel Based

- Negative Transformation
- Gamma Correction
- **Histogram equalization**

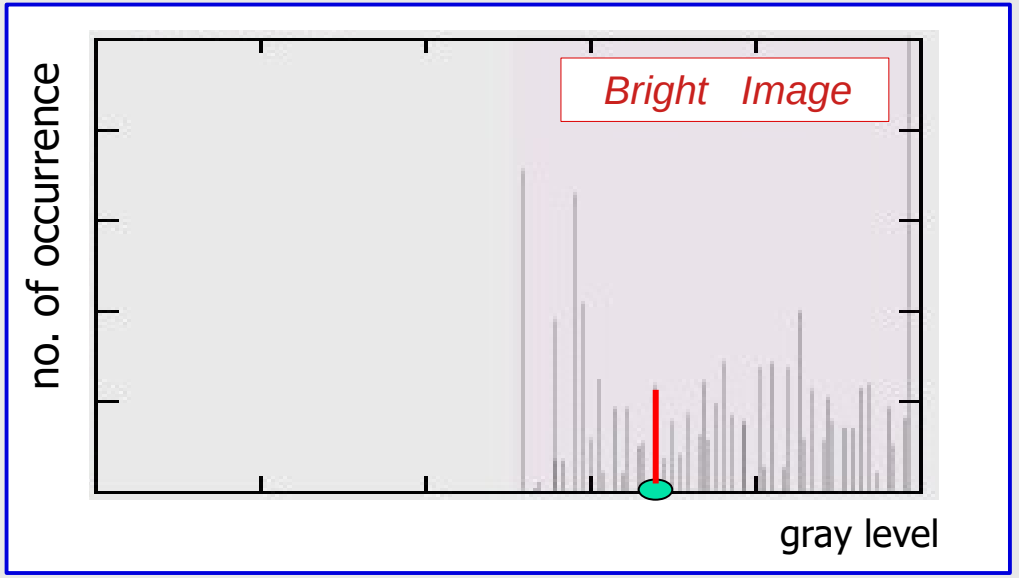
[2] Pixels Neighbors Based (Local Enhancement)

- Spatial filtering
  - Smoothing filters
  - Ordered statistics filters
  - Sharpening filters

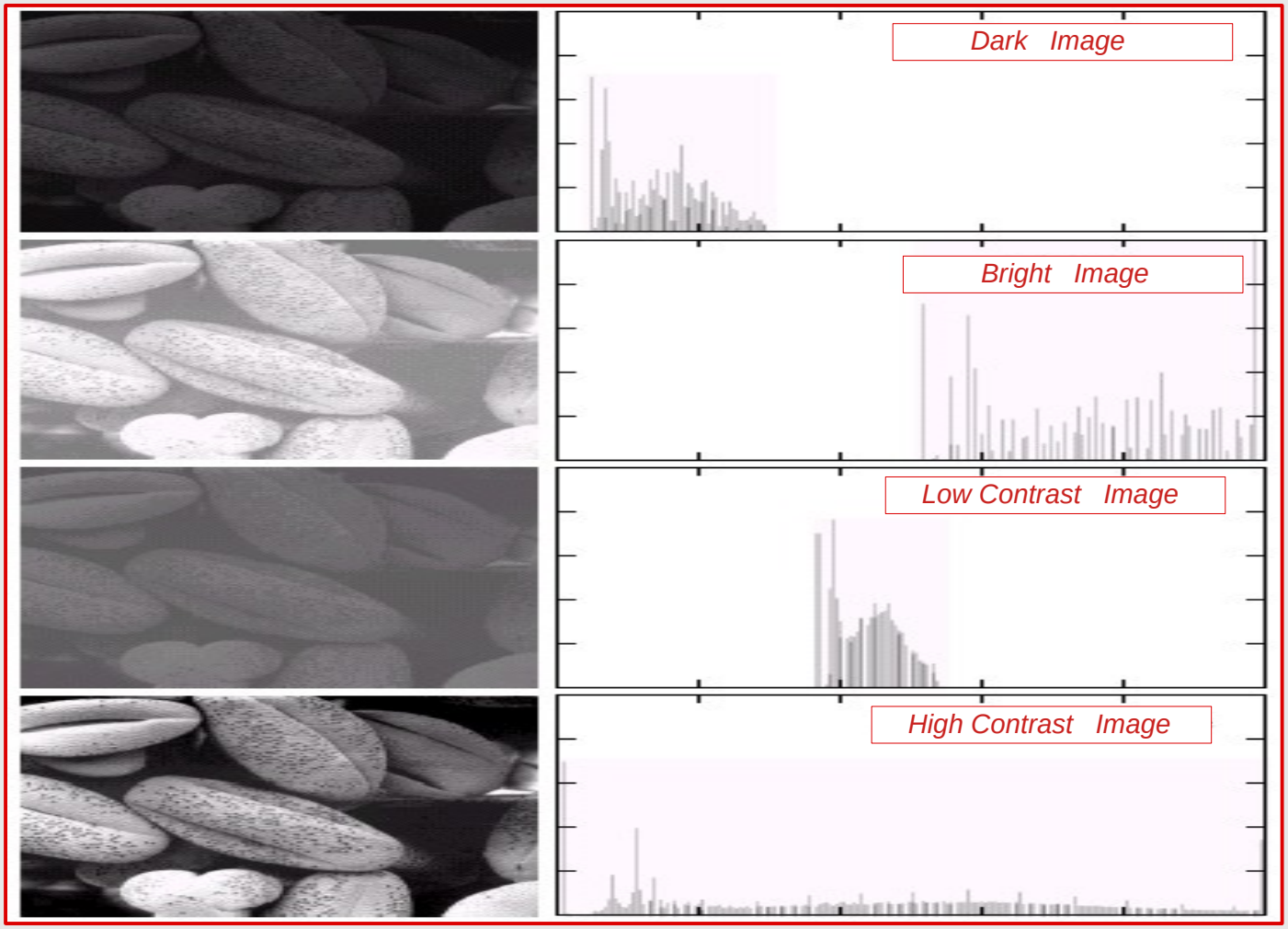
Histogram

Histogram is Statistics of the pixel gray-levels of an image

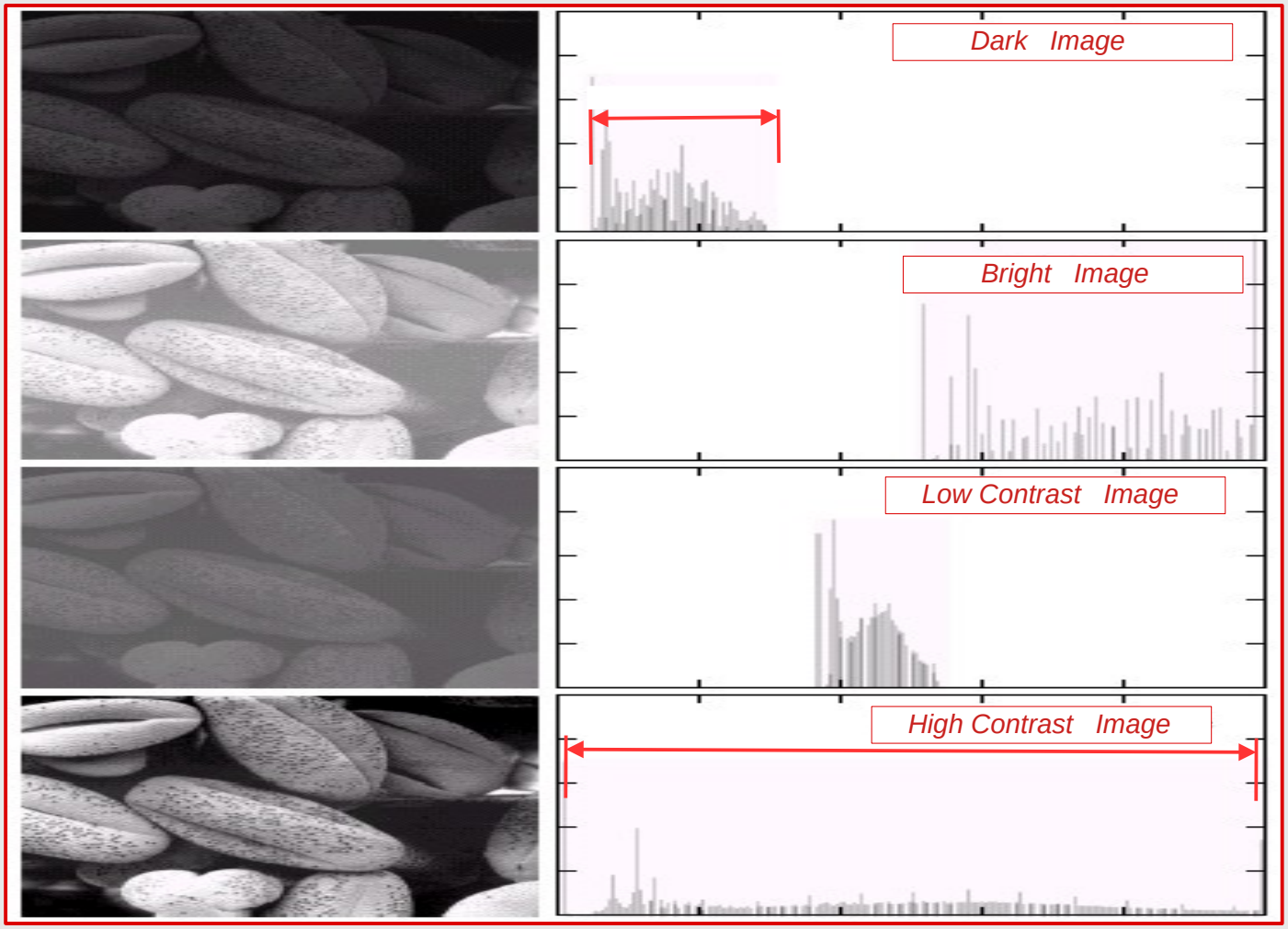
Histogram of Gray level K = ***Number of pixels*** *with gray level K in the image*



Obtain Contrast Information from Histogram



Obtain Contrast Information from Histogram

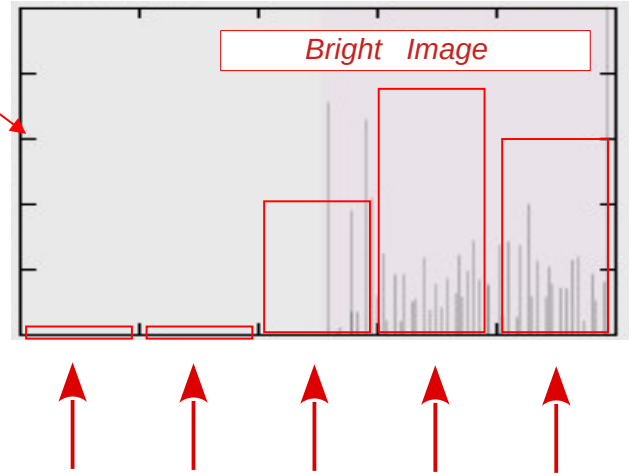


Histogram: implementation issue

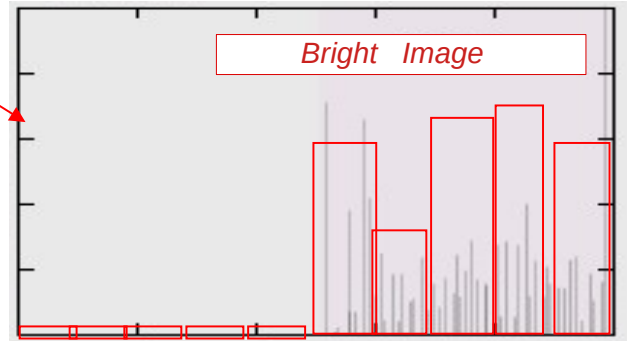
Divide the range of gray levels (ex. 0~255) into bins, Define:

- Number of bins
- Positions of bins

5 bins

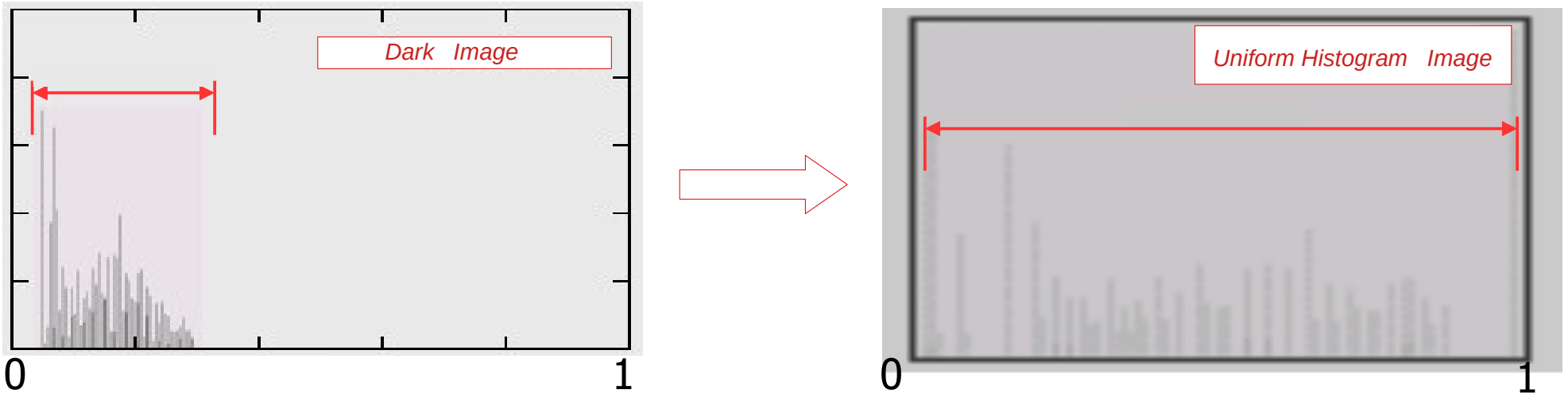


10 bins



[3] Histogram Equalization

- **Goal:** produce an uniform histogram
  - Find a transformation: s=T(r)



Histogram Equalization Example

Given an image of 50 pixels with 8 Intisity levels (from 0 to 7)

Intensity Level	0	1	2	3	4	5	6	7	*Max Value = 7*
Number of pixels	10	20	12	8	0	0	0	0	*Total 50 pixels*

Prob (Pixels with intensity =0) = 10/50 =0.2
Prob (Pixels with intensity =1) = 20/50 =0.4
Prob (Pixels with intensity =2) = 12/50 =0.24
Prob (Pixels with intensity =3) = 8/50 =0.16
Prob (Pixels with intensity =4) = 0/50 =0
Prob (Pixels with intensity =5) = 0/50 =0
Prob (Pixels with intensity =6) = 0/50 =0
Prob (Pixels with intensity =7) = 0/50 =0

For Pixels with intensity value = r, Max intensity value=N, Transform using T(r)

$$T(r)=\text{round}\left(N\sum_{i=0}^{r}p(i)\right)$$

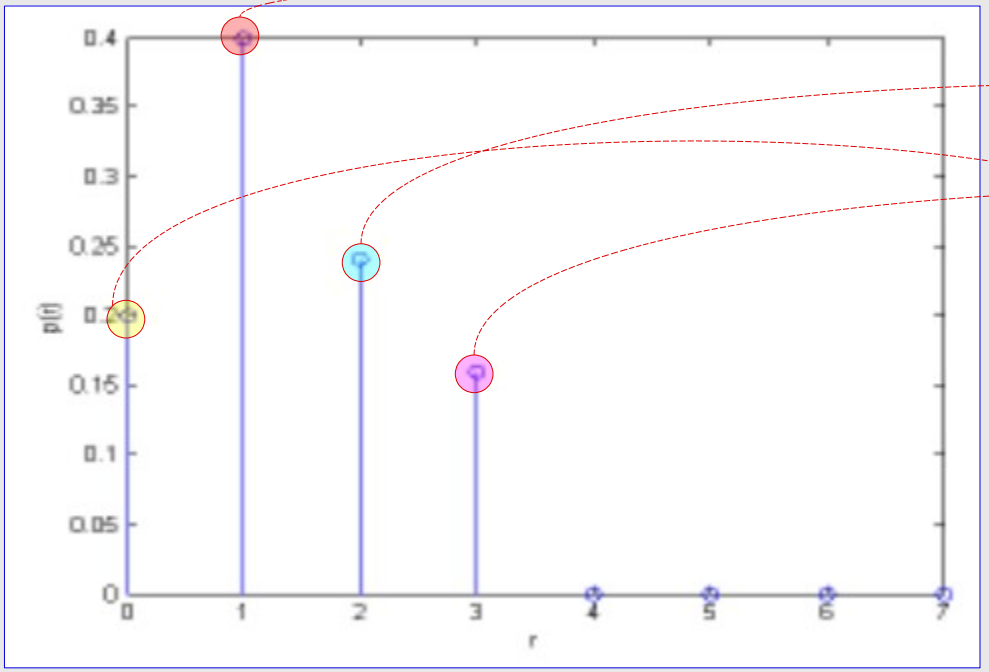
$$T(0)=\text{round}\left(7*p(0)\right)=\text{round}\left(7*0.2\right)=1$$
$$T(1)=\text{round}\left(7*\left(p(0)+p(1)\right)\right)=\text{round}\left(7*\left(0.2+0.4\right)\right)=\text{round}\left(7*0.6\right)=4$$
$$T(2)=\text{round}\left(7*\left(p(0)+p(1)+p(2)\right)\right)=\text{round}\left(7*\left(0.2+0.4+0.24\right)\right)=\text{round}\left(7*0.84\right)=6$$
$$T(3)=\text{round}\left(7*\left(p(0)+p(1)+p(2)+p(3)\right)\right)=\text{round}\left(7*\left(0.2+0.4+0.24+0.16\right)\right)=\text{round}\left(7*1\right)=7$$

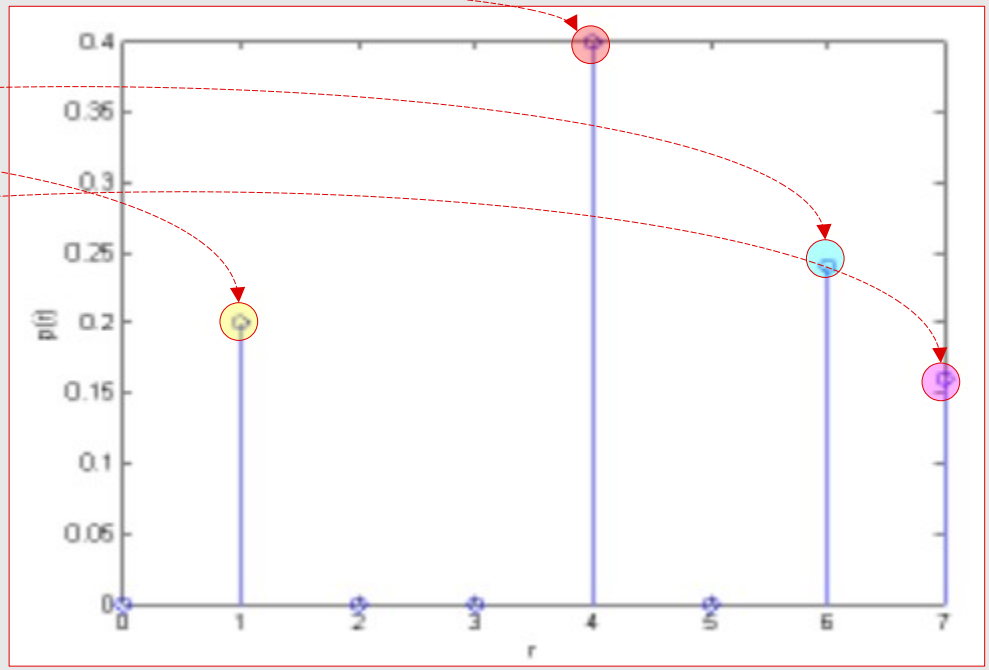
Intensity Levels Mapping

Original Intensity Level	0	1	2	3	4	5	6	7
New Intensity Level	1	4	6	7	N/A	N/A	N/A	N/A

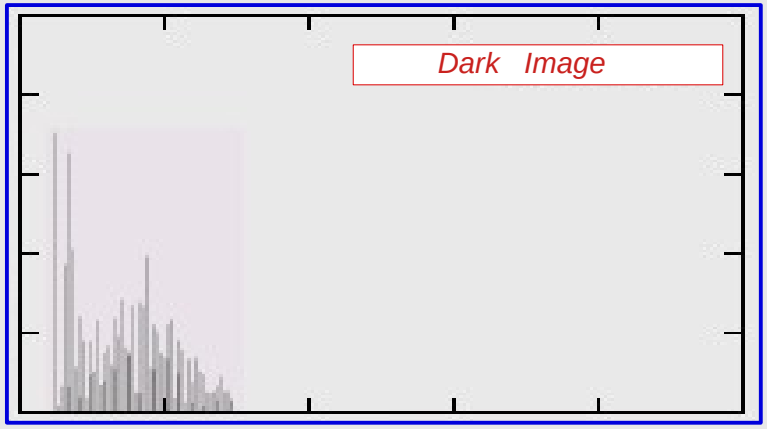
Histogram Equalization Example

Intensity Levels Mapping								
Original Intensity Level	0	1	2	3	4	5	6	7
New Intensity Level	1	4	6	7	N/A	N/A	N/A	N/A
Number of Pixels	10	20	12	8	0	0	0	0



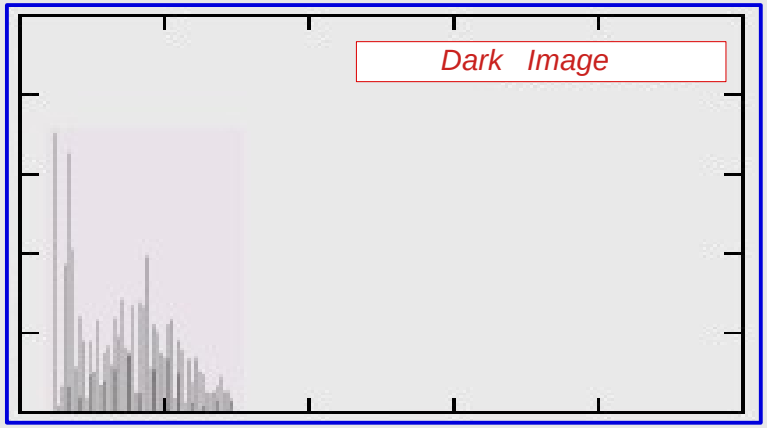


Example: Histogram Equalization



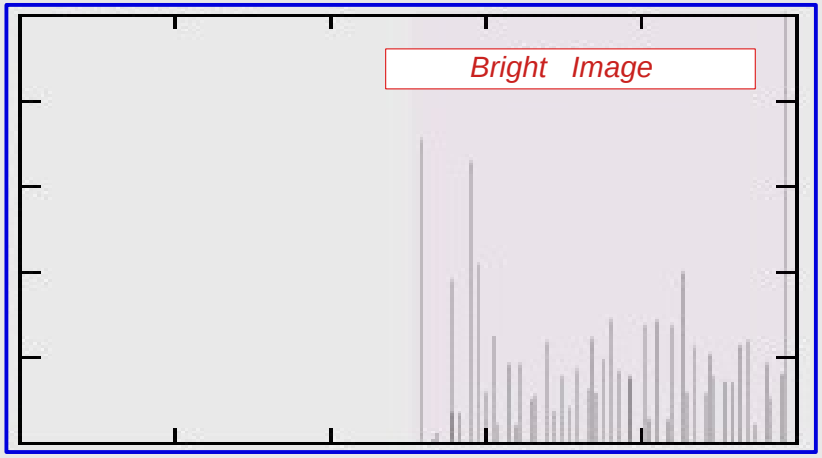
Example: Histogram Equalization

After Equalization



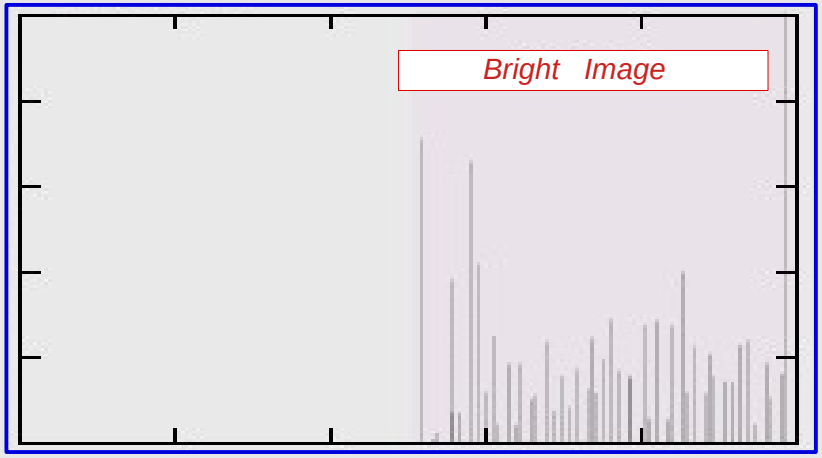
After Equalization

Example: Histogram Equalization

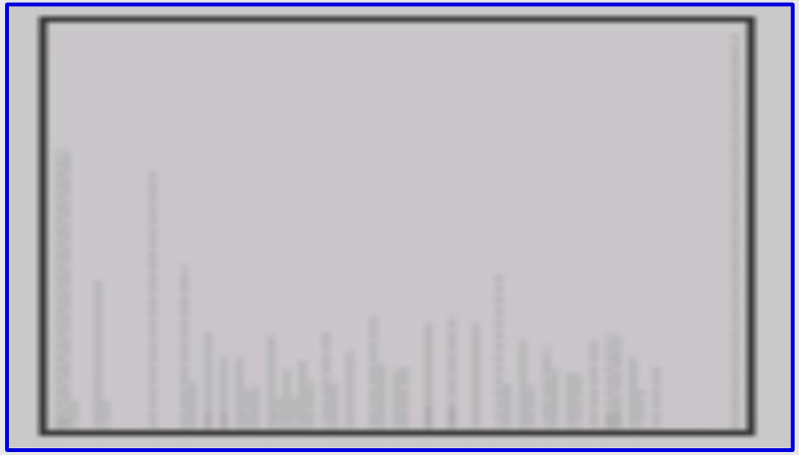


Example: Histogram Equalization

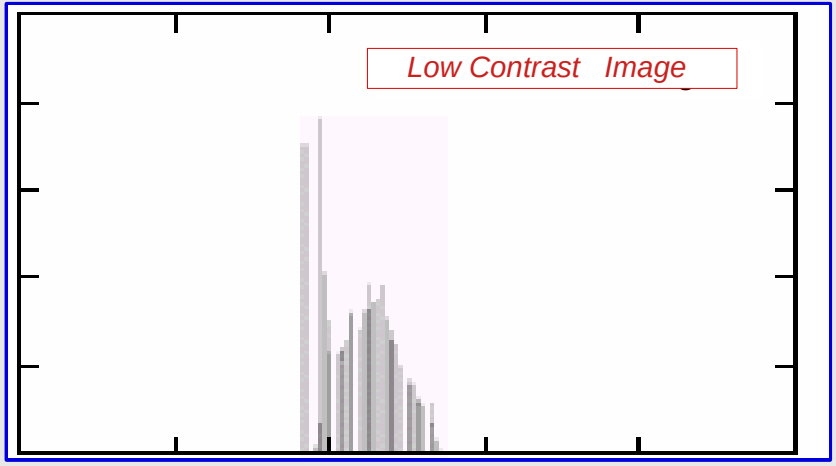
After Equalization



After Equalization

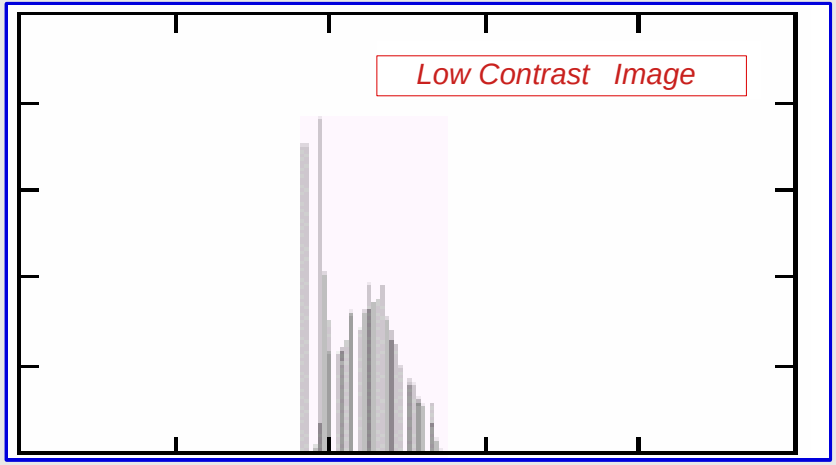


Example: Histogram Equalization



Example: Histogram Equalization

After Equalization



After Equalization

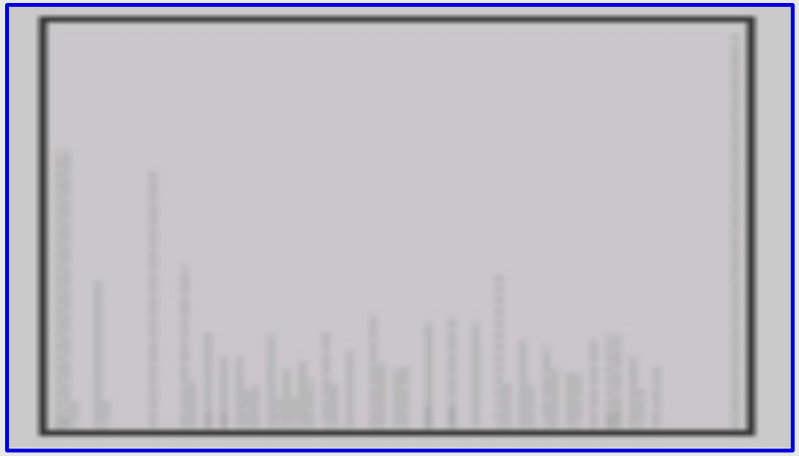


Image Enhancement

[1] Pixel Based

- Negative Transformation
- Gamma Correction
- Histogram equalization

[2] Pixels Neighbors Based (Local Enhancement)

Spatial filtering

- Smoothing filters
- Ordered statistics filters
- Sharpening filters

Basics of spatial filtering

g(x,y)=**T**[f(x,y)]

T operates on neighborhood Pixels

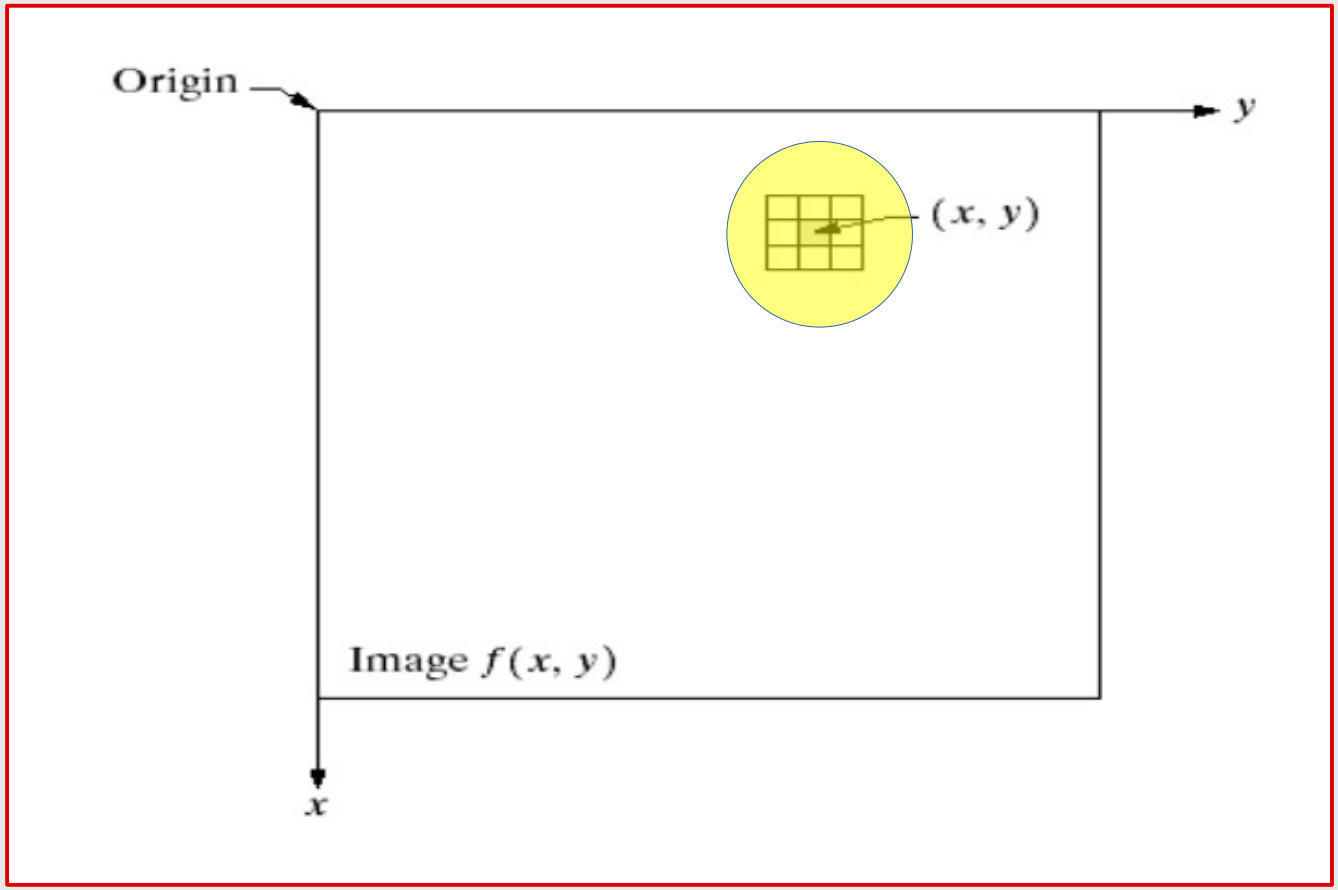


Image Enhancement

[1] Pixel Based

- Negative Transformation
- Gamma Correction
- Histogram equalization

[2] Pixels Neighbors Based (Local Enhancement)

- **Spatial filtering**
  - **Smoothing filters**
  - Ordered statistics filters
  - Sharpening filters

Smoothing spatial filtering

Used for blurring and for noise reduction

Smoothing linear filters (Average filters, low-pass filters, ...)

$\frac{1}{9} \times$

1	1	1
1	1	1
1	1	1

average filter

$\frac{1}{16} \times$

1	2	1
2	4	2
1	2	1

weighted filter

original

3x3

noise reduced

5x5

9x9

small area blended into background

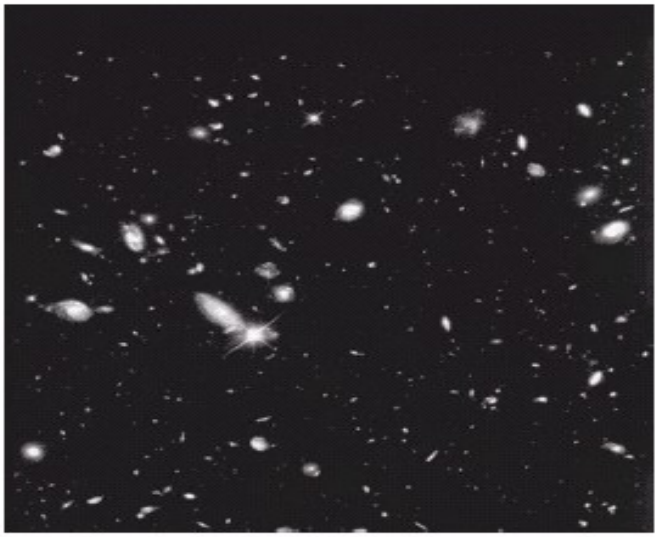
15x15

35x35

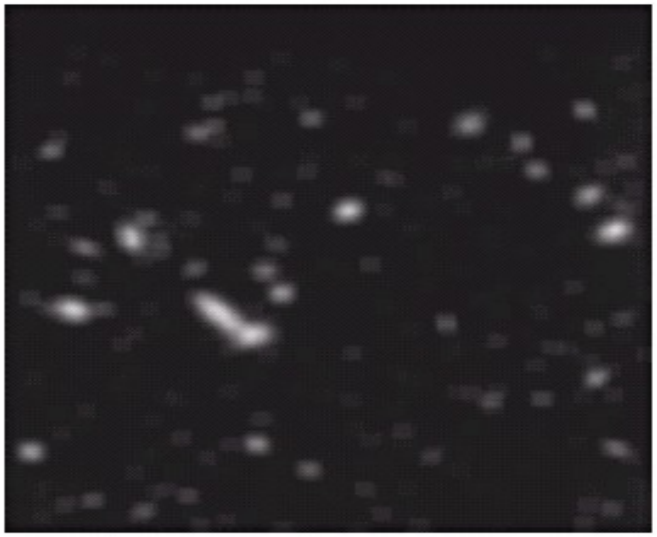
border effects

Smoothing: example

15x15 smoothing ⇨ thresholding ⇨



(a) Image From Hubble Space Telescope (NASA)



(b) Image Processed by 15x15 Averaging Mask
(Noise Removed – Blurred Image)

(c) Results After Binarization (Thresholding)

Image Enhancement

[1] Pixel Based

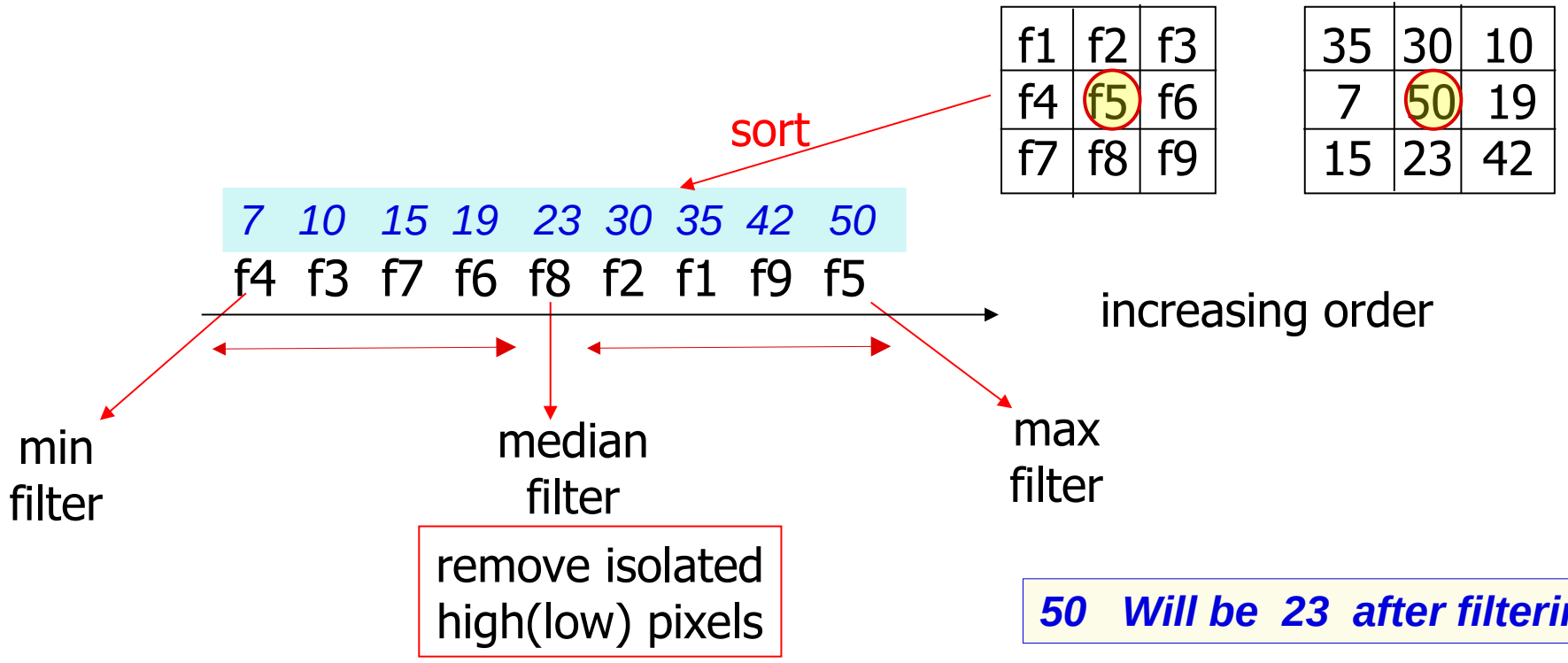
- Negative Transformation
- Gamma Correction
- Histogram equalization

[2] Pixels Neighbors Based (Local Enhancement)

- **Spatial filtering**
  - Smoothing filters
  - **Ordered statistics filters**
  - Sharpening filters

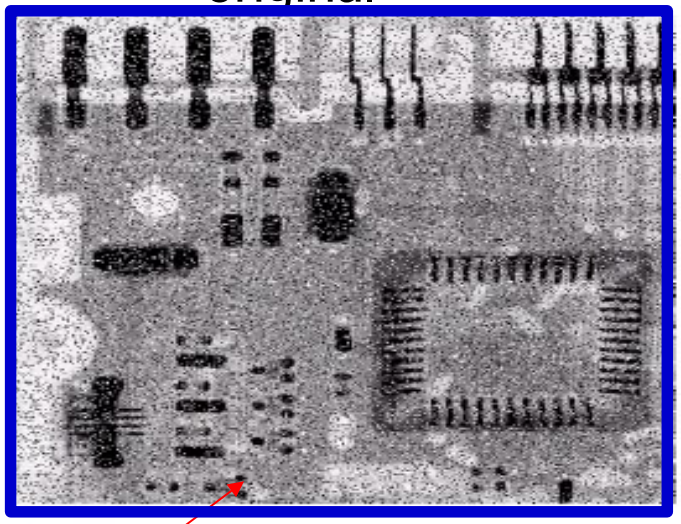
Order-statistics filters

Based on Pixel neighborhood

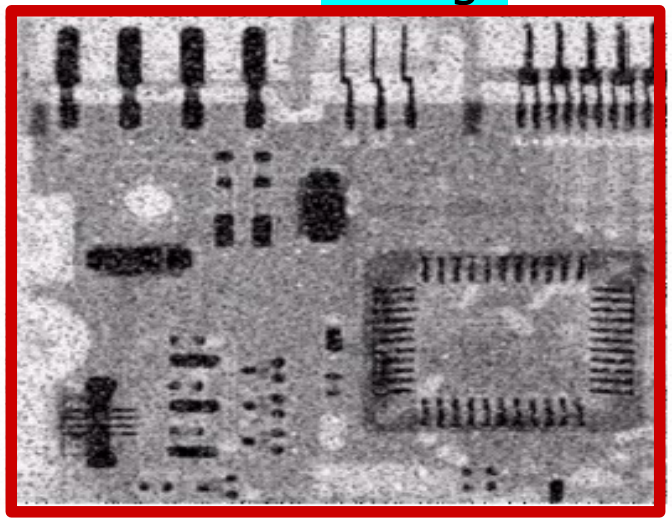


Median filter: example

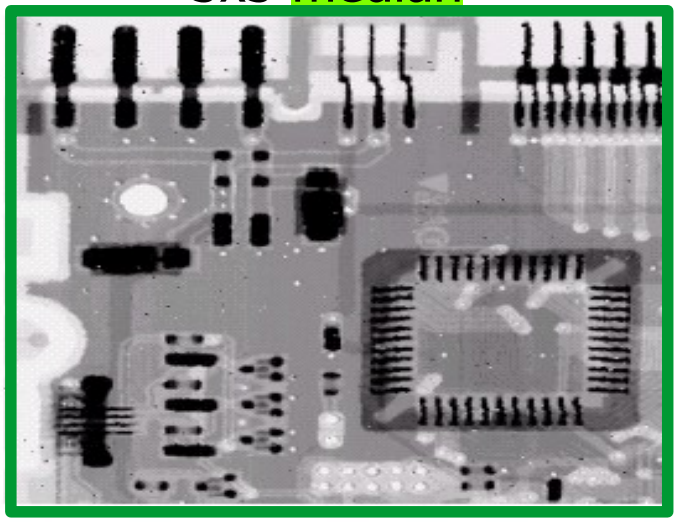
original



3x3 average



3x3 median



salt-and-pepper noise

Image Enhancement

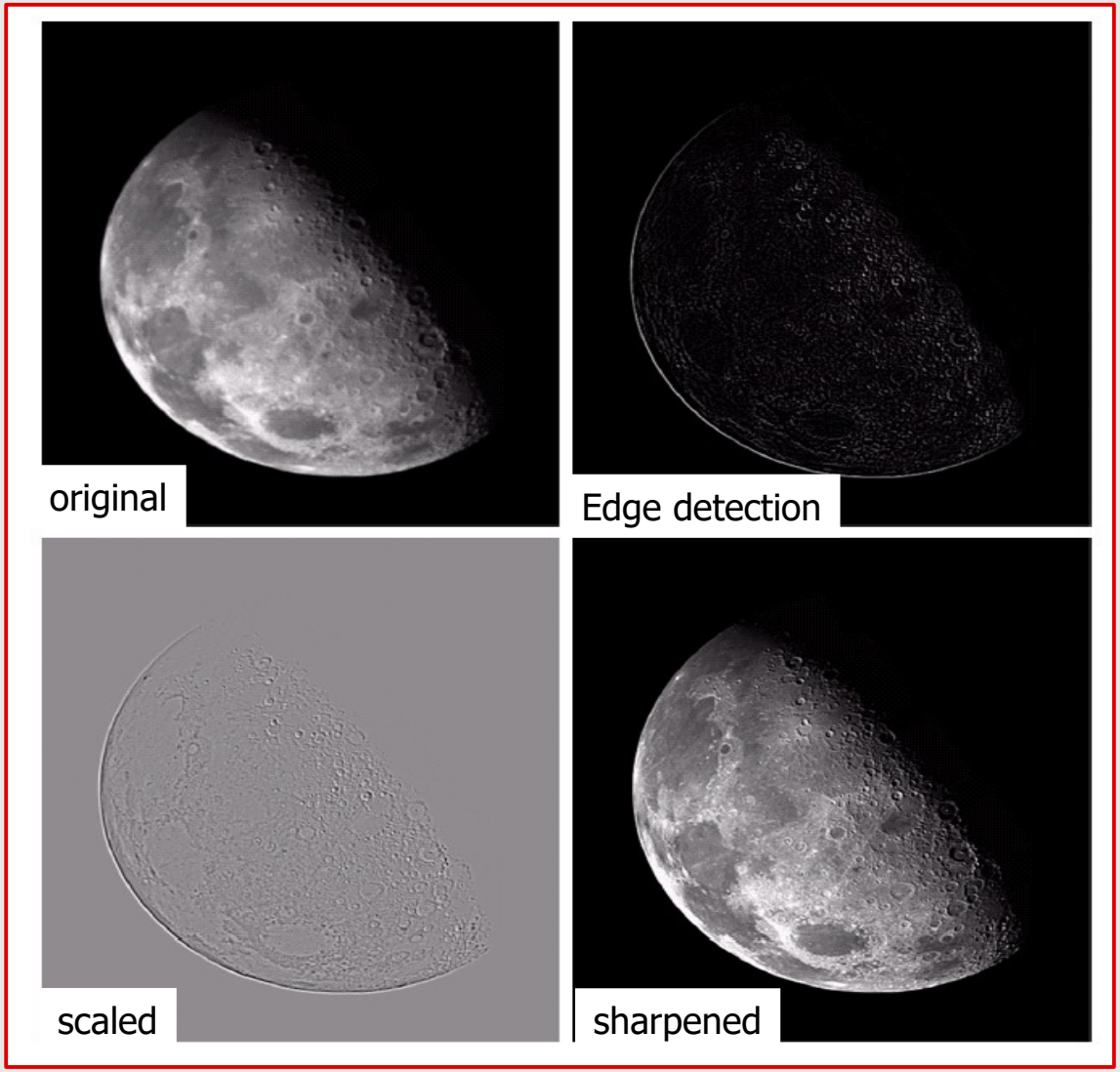
[1] Pixel Based

- Negative Transformation
- Gamma Correction
- Histogram equalization

[2] Pixels Neighbors Based (Local Enhancement)

- **Spatial filtering**
  - Smoothing filters
  - Ordered statistics filters
  - **Sharpening filters**

Sharpening filtering: example



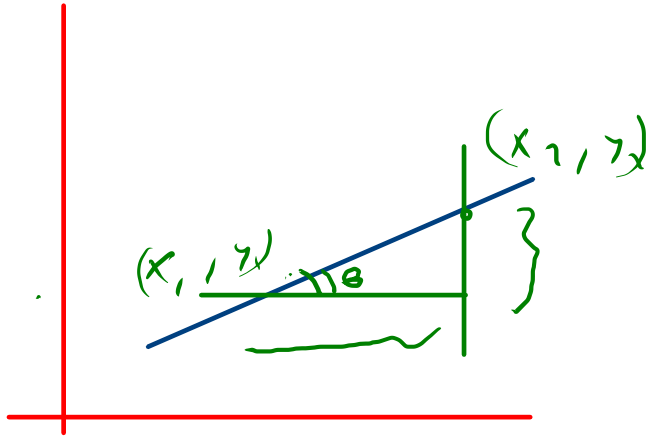
Sharpening spatial filters

Goal

[1] Highlight fine details in an image

Spatial differentiation

Digital implementation = differences

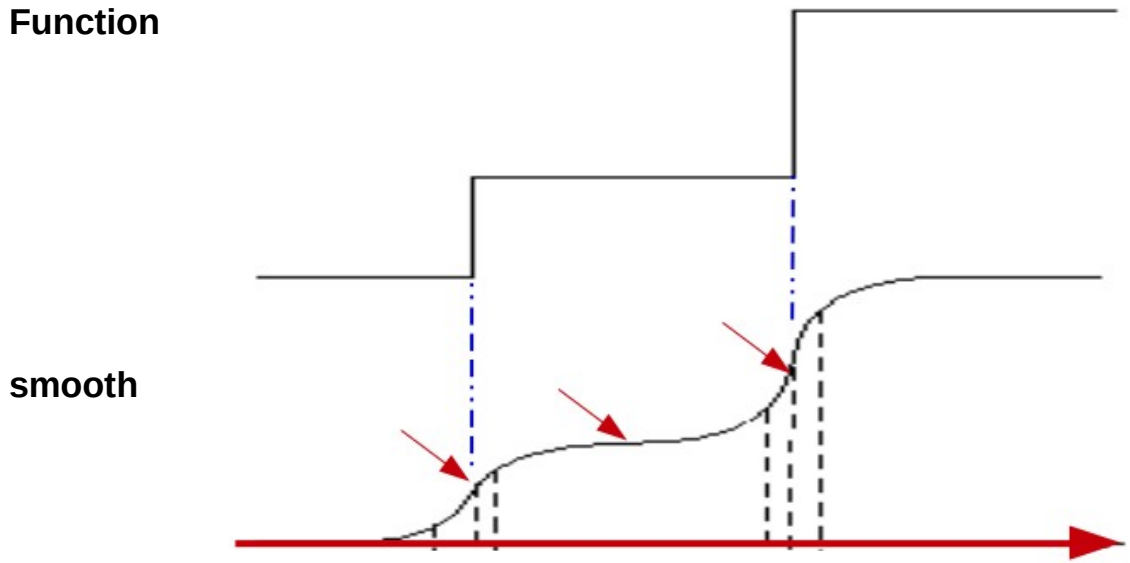


[2] De-emphasize regions with slowly varying gray levels

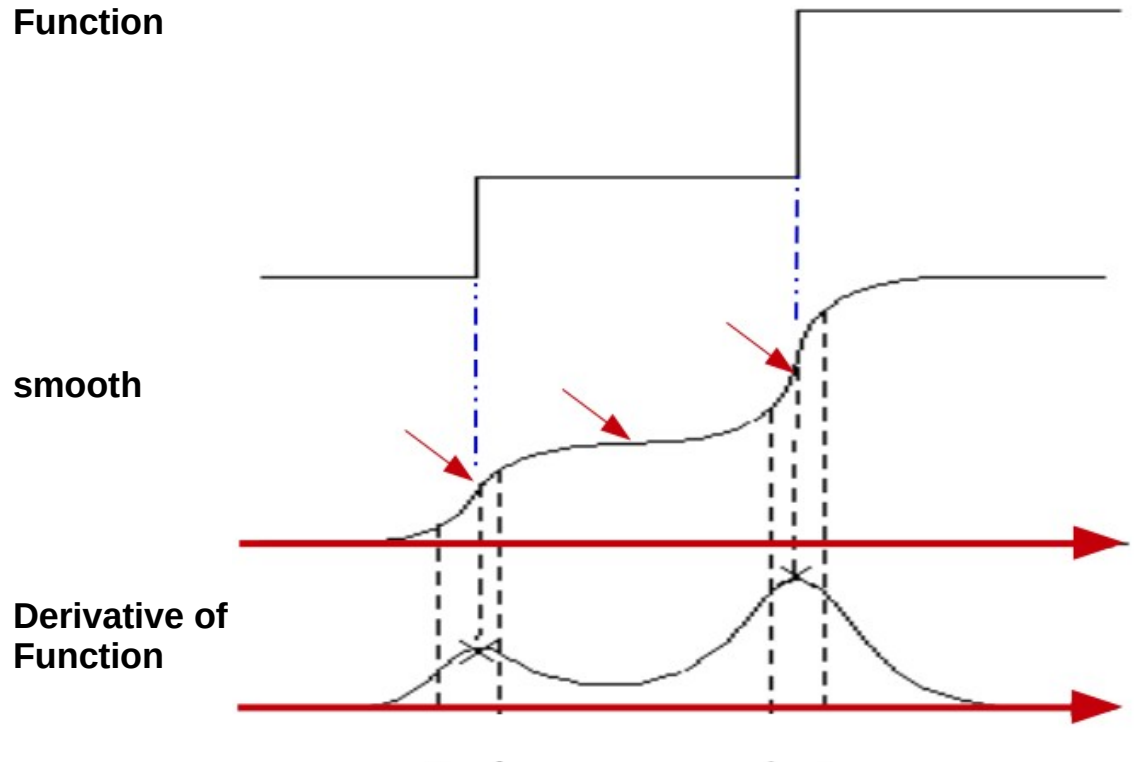
(Almost Smoothing)

$$\tan\theta = \frac{(y_2 - y_1)}{(x_2 - x_1)} = \frac{\Delta y}{\Delta x}$$

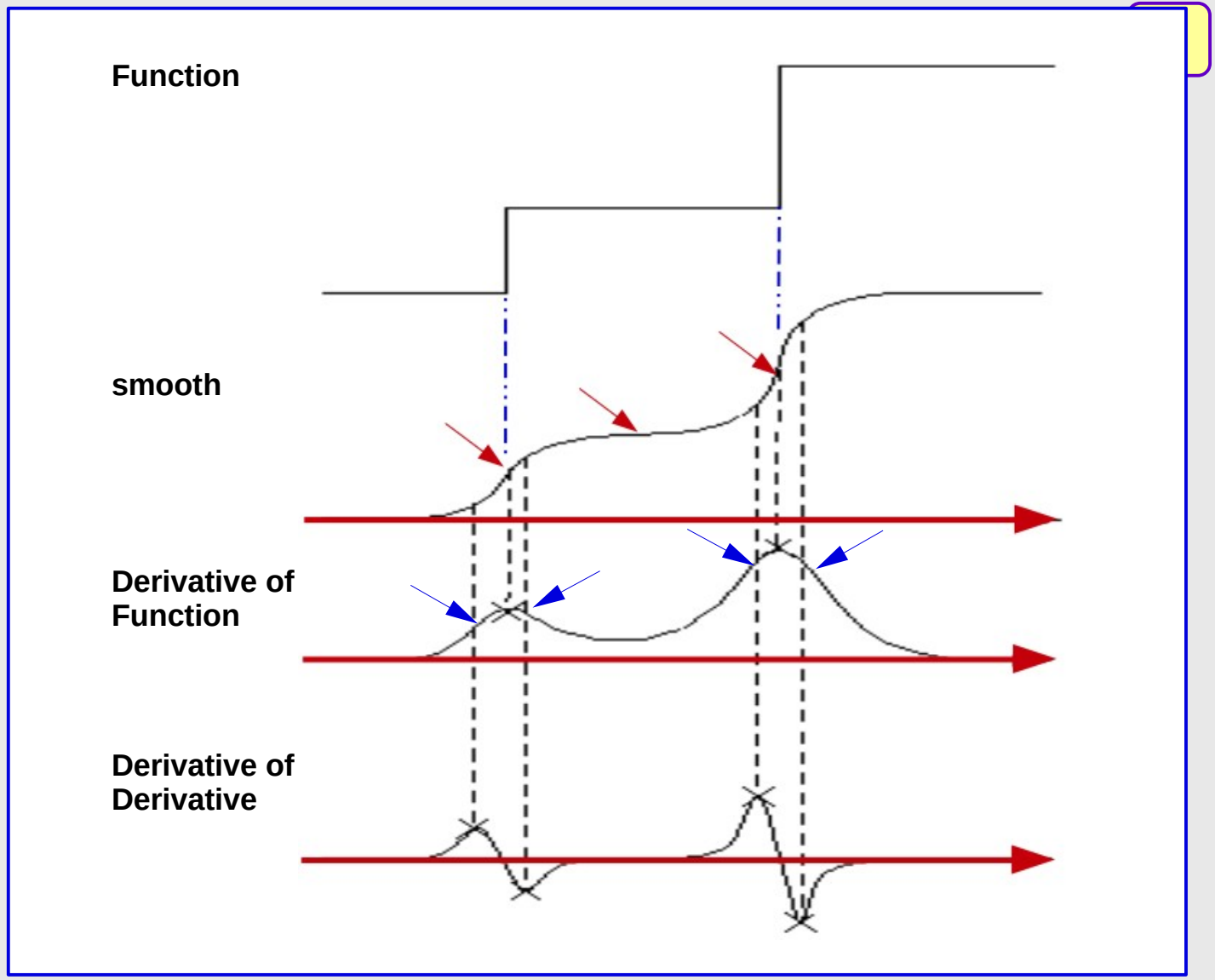
Smoothing
(to be differentiable)



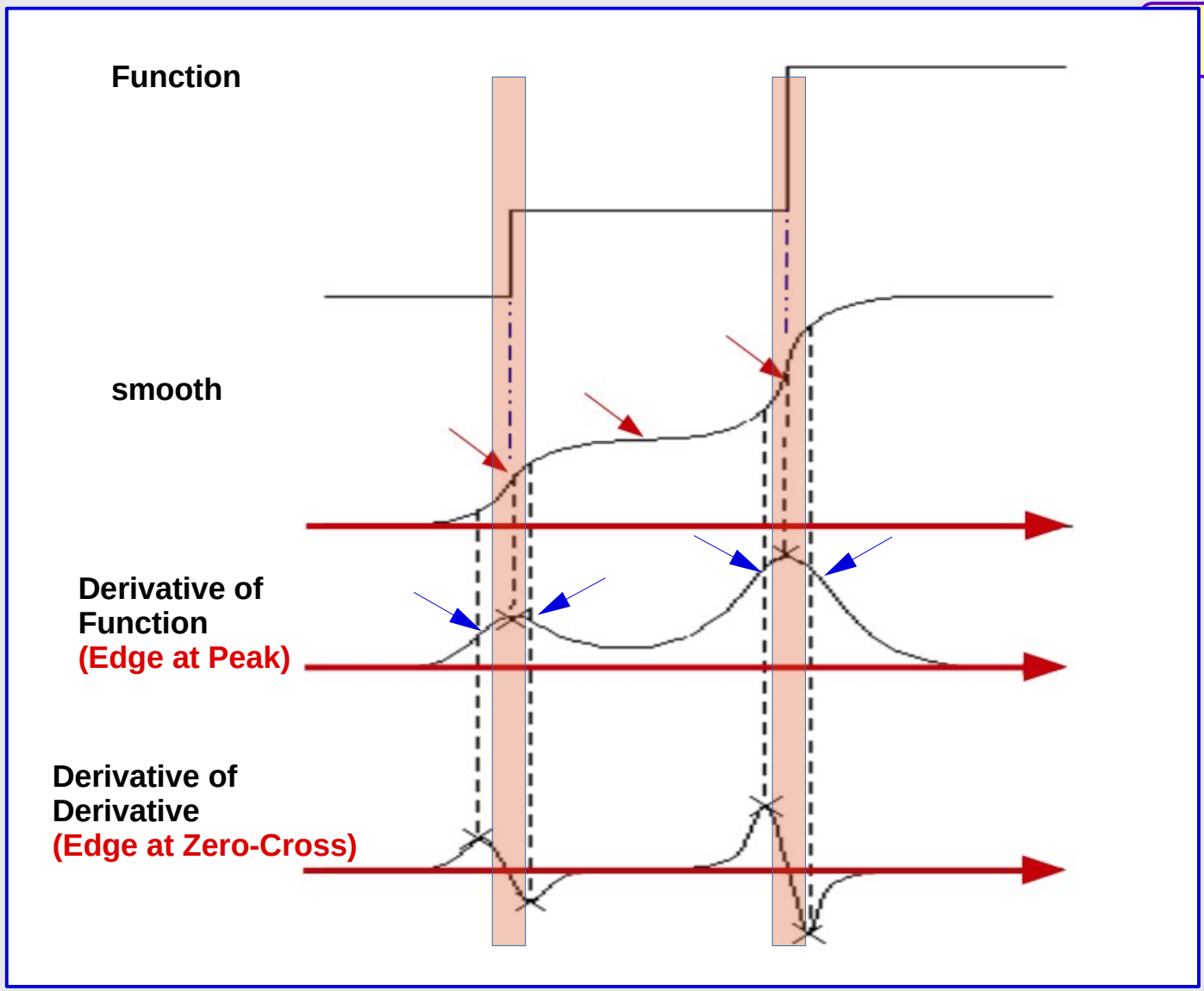
First Derivative of Smoothed Function



Edges position: between Min and Max points (of 2nd derivative)



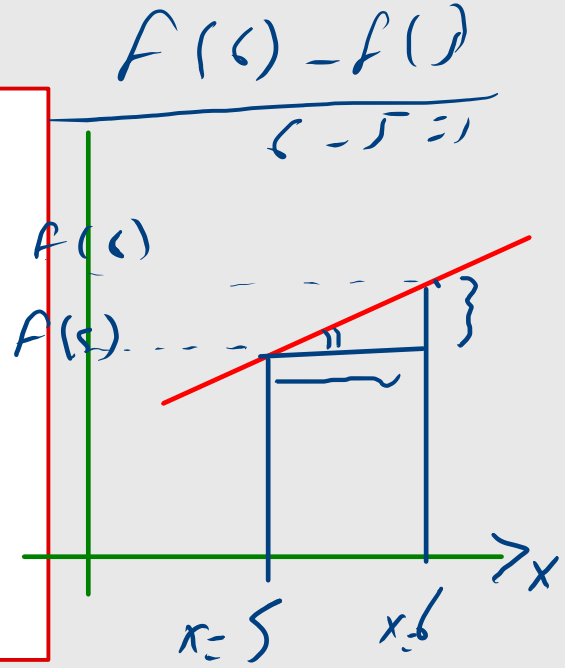
Location of the Edges



Sharpening: foundation

First and second-order derivatives in digital form = **difference**

$$\begin{cases} \dfrac{\partial f}{\partial x} = [f(x+1) - f(x)]/1 \\ \dfrac{\partial^2 f}{\partial x^2} = [f(x+1) - f(x)] - [f(x) - f(x-1)] \\ \qquad = f(x+1) + f(x-1) - 2f(x) \end{cases}$$



2nd Derivatives of Images [Laplacian]

$$\nabla^2 f = [f(x+1,y)+f(x-1,y)-2f(x,y)] + [f(x,y+1)+f(x,y-1)-2f(x,y)]$$

$$= [f(x+1,y)+f(x-1,y)+f(x,y+1)+f(x,y-1)]-4f(x,y)$$

f(x-1,y-1)	f(x-1,y)	f(x-1,y+1)
f(x,y-1)	f(x,y)	f(x,y+1)
f(x+1,y-1)	f(x+1,y)	f(x+1,y+1)

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

For one variable:

$$\frac{\partial^2 f}{\partial x^2} = f(x+1)+f(x-1)-2f(x)$$

$$\frac{\partial^2 f}{\partial y^2} = f(y+1)+f(y-1)-2f(y)$$

Definition of 2nd derivatives as filter mask

90⁰ rotation invariant

0	1	0
1	−4	1
0	1	0

1	1	1
1	−8	1
1	1	1

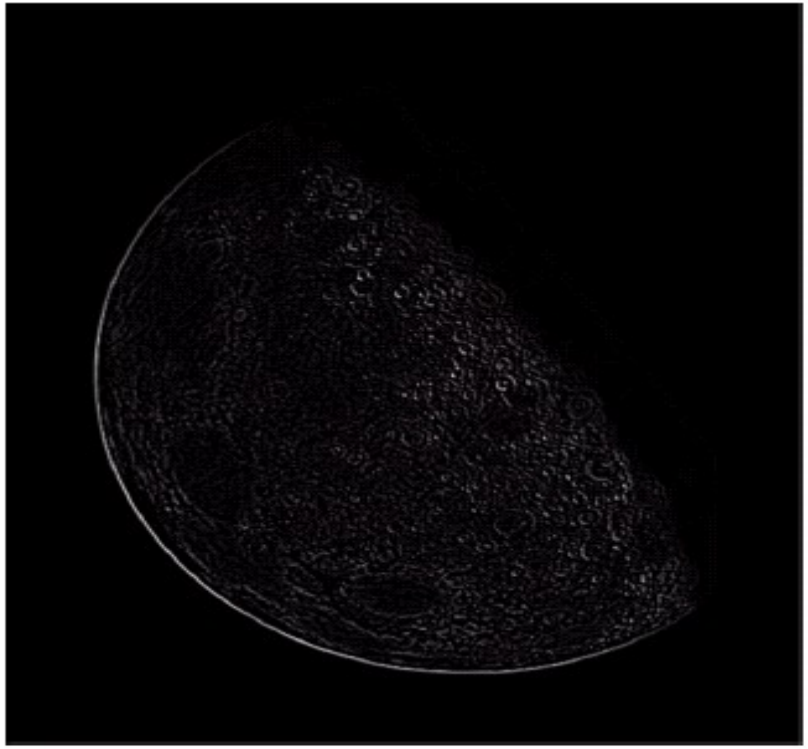
0	-1	0
-1	4	- 1
0	- 1	0

- 1	-1	- 1
- 1	8	-1
- 1	-1	- 1

45⁰ rotation invariant (include Diagonals)

Laplacian filtering: example

original



Laplacian filtered image

Sharpened result = Original + Laplacian filtered image

$$g(x,y) = \begin{cases} f(x,y) - \nabla^2 f(x,y) & \text{if center coeff. of the mask is negative} \\ f(x,y) + \nabla^2 f(x,y) & \text{if center coeff. of the mask is positive} \end{cases}$$

original image

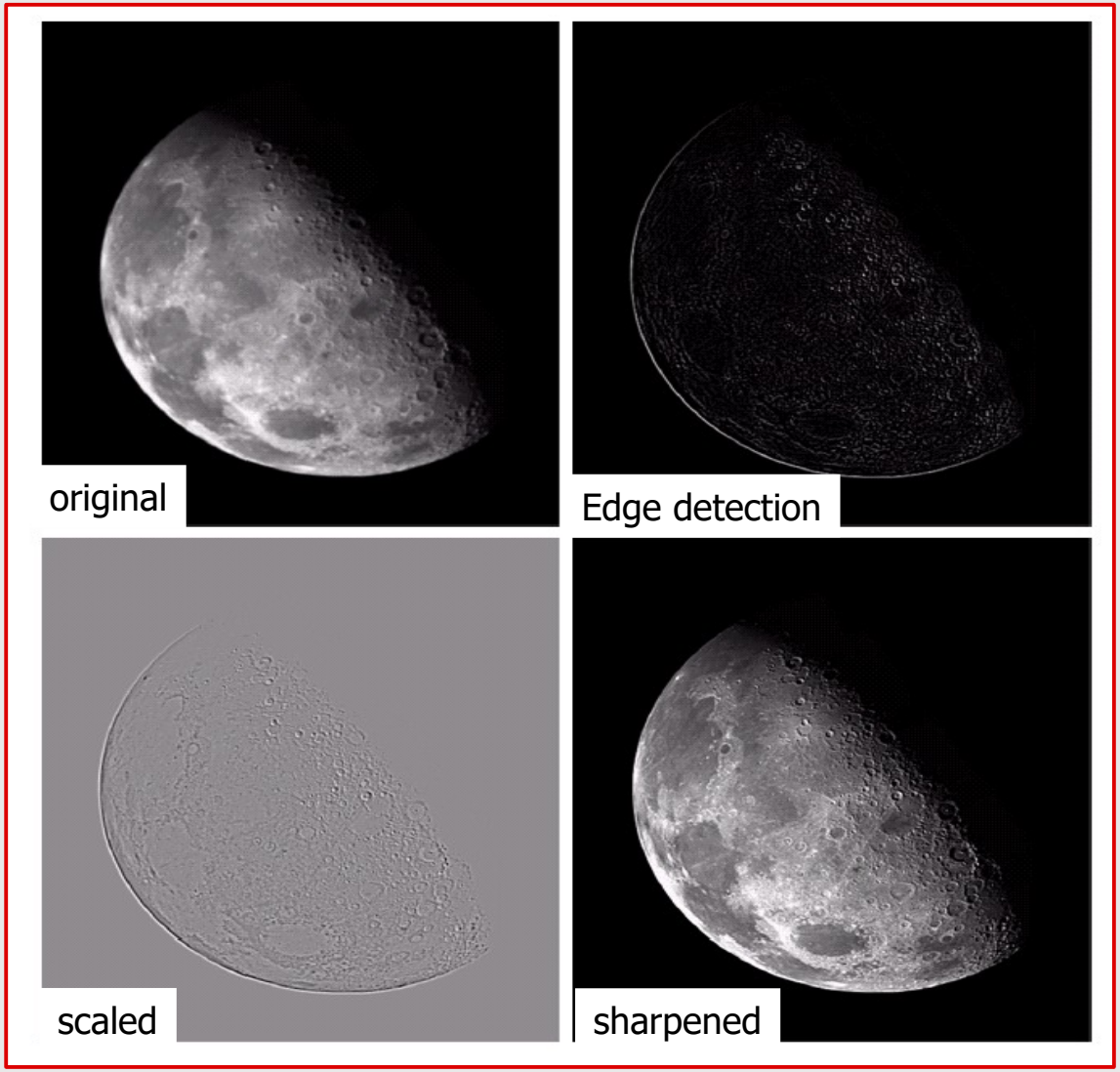
filtered image

resulting mask

0	−1	0
−1	5	−1
0	−1	0

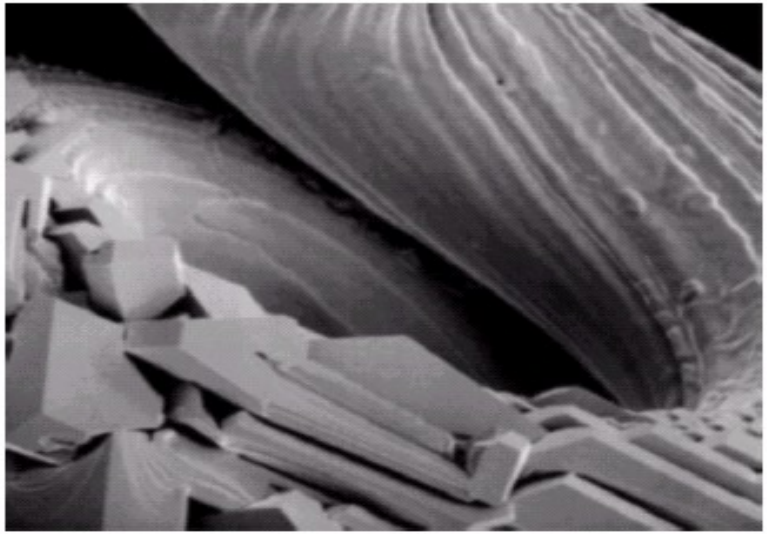
−1	−1	−1
−1	9	−1
−1	−1	−1

Laplacian filtering: example



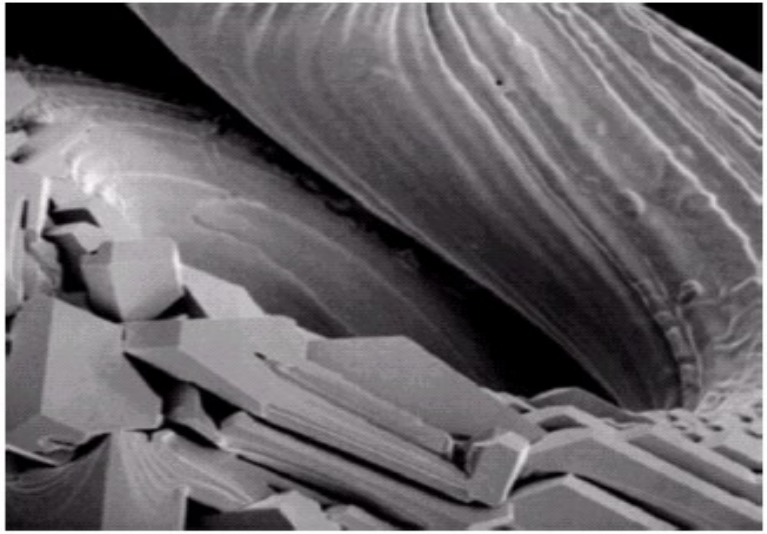
0	−1	0
−1	5	−1
0	−1	0

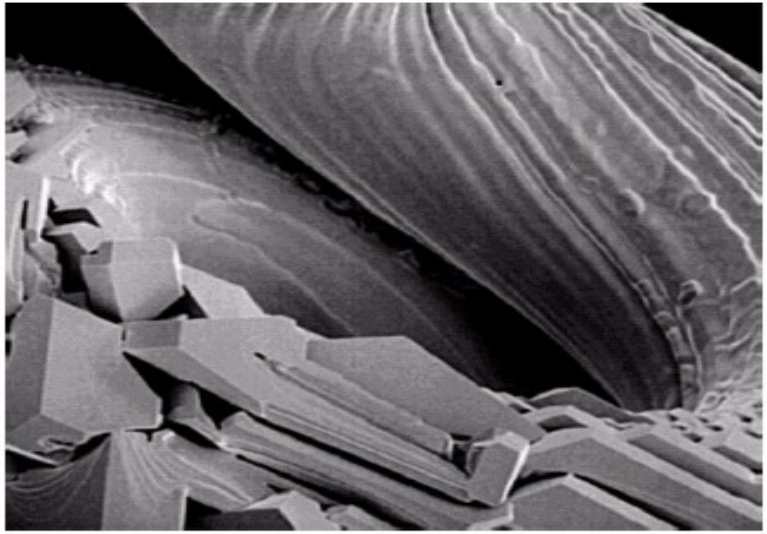
−1	−1	−1
−1	9	−1
−1	−1	−1



original

Cyan mask (90°)





Orange mask (45°)